

LECTURE – 7

LAB ASSIGNMENT #2

Running time with longer records

1. Complete the remaining part of Assignment #1.

2. Carry out the following extension of Assignment #1:

(a) Append satellite data of length N bytes to the integer key field, for N = 50, 100, 500, 1000 ... Plot the resulting running times for insertion sort and merge sort. [The content of these N bytes is irrelevant, since we are interested only in the increase in running time caused by moving long records. You may select any one character (say 'X') and repeat it N times.]

(b) OPTIONAL Append to the key field only a pointer to satellite data. So when a record has to move, only the fixed length pointer moves, but not the satellite data itself. Compare the running time with that found in (a).

HEAPSORT (continued from Lecture - 6)

Running time of MAX-HEAPIFY:

$T(n) \leq T(2n/3) + \Theta(1)$ // Since **recursion goes to at most one sub-tree**
// of the node i for which the function is called.

By Case 2 of the Master theorem ($a=1$, $b=3/2$), the solution is $O(\lg n)$.

Now we use this function to design the main HEAPSORT algorithm.

Step 1: Convert unsorted array $A[1 \dots n]$ into a max-heap.

Key insight: [CLRS]

... by definition, array elements $A[\text{FLOOR}(n/2)+1 \dots n]$ are leaf nodes and, therefore, each of these is a 1-element heap to begin with.

So work is needed only on elements $A[1 \dots \text{FLOOR}(n/2)]$. That is what the following function does:

```

BUILD-MAX-HEAP (A)
  HEAP-SIZE (A) = LENGTH (A) ;
  FOR I = FLOOR (LENGTH (A) / 2) , 1, -1  // Note -1
    MAX-HEAPIFY (A, I) ;

```

Running time is $O(n)$.

Note that this is tighter than the easily obtained higher bound of $O(n \lg n)$. The reason is that, as we go one level higher up the tree, the number of nodes at that level reduces by half. The tighter upper bound therefore depends on this summation:

$$\sum_{h=0}^{\infty} (n/2^h), \quad \text{where } h \text{ represents the height of a node.}$$

This is $\Theta(1)$, using a property of geometric series. Simple logic then yields the tighter running time $O(n)$ for **BUILD-MAX-HEAP** [CLRS]. But, as seen below, this function is **outside the main loop** of HEAPSORT.

Step 2: Use the built max-heap to sort the n array elements.

```

HEAPSORT (A)
  BUILD-MAX-HEAP (A) ;
  FOR I = LENGTH (A) , 2, -1
    // Move the maximum element to the end
    EXCHANGE ( A[1] , A[I] ) ;
    HEAP-SIZE[A] = HEAP-SIZE[A] - 1 ;
    MAX-HEAPIFY (A, 1) ;

```

The way in which this algorithm is implemented in [CLRS], the maximum element is being moved to the end, then effectively removed from A , and the process repeated on the remaining elements of A .

Running time is seen easily to be $O(n \lg n)$.

Quick summary:

The max-heap data structure can also be used to implement a **priority queue**. Heap functions needed are then e.g.: INSERT(S, x), MAXIMUM(S), EXTRACT-MAX(S), INCREASE-KEY(S, x, k) ... etc, where S is the relevant **set**.

Min-heap can be useful in implementing **event-driven, time-based systems**.